Cyber Impulse
SOFTWARE SOLUTIONS

AI-POWERED MULTI-VENDOR FASHION MARKETPLACE

Technical Architecture & Developer Implementation Guide

Node.js ESM | Next.js | Flutter | Python AI | English + Italian

Prepared by	Cyber Impulse Software Solutions
Document type	Internal technical implementation standard
Version	1.0
Date	09 September 2026

CONFIDENTIAL - DEVELOPMENT TEAM

00 Purpose, Scope and Non-Negotiable Rules

This guide is the shared technical contract for backend, web, mobile and AI development.

The project is a multi-vendor fashion marketplace with customer web and mobile channels, a vendor portal, a full administration panel, English and Italian interfaces, AI-assisted body measurement, size recommendation and virtual clothing try-on.

> **Core architecture rule**
>
> Flutter and Next.js call Node.js only. Node.js owns public APIs, authentication, business rules, data and AI job orchestration. Python AI and cloud AI providers are private implementation details behind Node.js.

Final stack

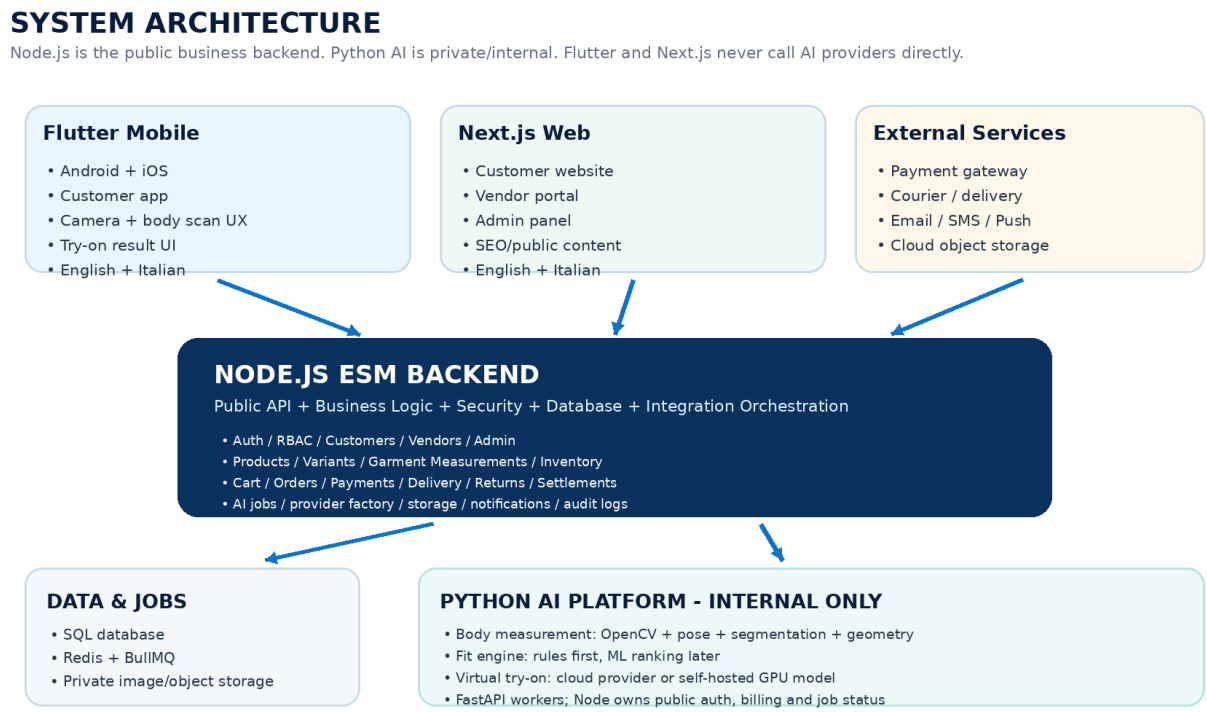
Layer	Standard
Backend	Node.js ESM + Express.js + Sequelize + SQL + Redis/BullMQ
Web	Next.js App Router - customer website, vendor portal and admin panel
Mobile	Flutter - Android and iOS customer application
AI	Python + FastAPI + OpenCV + PyTorch/vision models; cloud provider adapters
Languages	English (en) and Italian (it)
Storage	Private object storage for product, body-scan and try-on images
Integration	REST/HTTPS; asynchronous AI jobs; webhooks for payment/delivery

Non-negotiable development rules

- No API keys, cloud credentials or provider secrets in Flutter or browser-side Next.js code.
- No direct Flutter -> Python or Next.js -> Python calls.
- No heavy generative AI or exact body measurement inference inside Flutter.
- All third-party AI access is wrapped behind backend provider adapters.
- All production schema changes use Sequelize migrations; production model sync is disabled.
- All UI strings are localized through English/Italian resource files; do not hard-code user-facing copy throughout components.
- All AI jobs have auditable status, cost/provider metadata, retries and failure handling.

01 System Architecture

Four repositories/services with clear ownership and one public application backend.

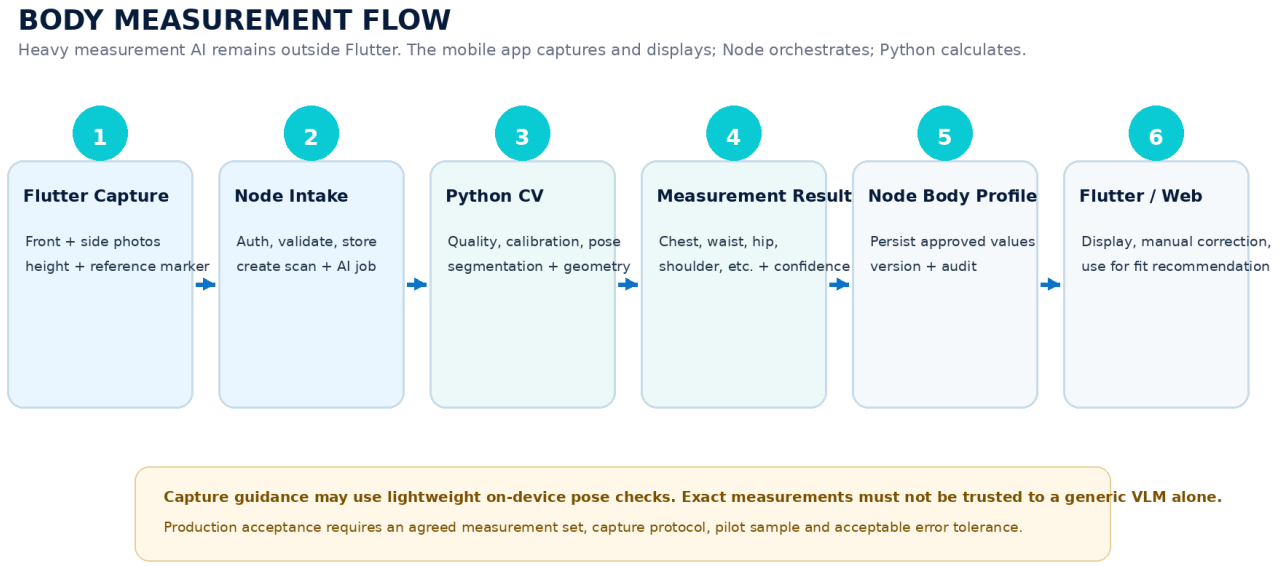


Communication boundaries

Caller	Allowed target	Not allowed
Flutter	Node.js public API	Python AI, Gemini, Vertex, OpenAI directly
Next.js	Node.js public API	Python AI or provider secrets in client code
Node.js	SQL, Redis, storage, payment/delivery providers, Python AI, cloud AI	Provider logic scattered inside controllers
Python AI	Internal Node callbacks / private storage as approved	Public customer authentication or marketplace business logic

02 Body Measurement Flow

The mobile application provides capture UX; the Python service performs the measurement computation.



Recommended result contract

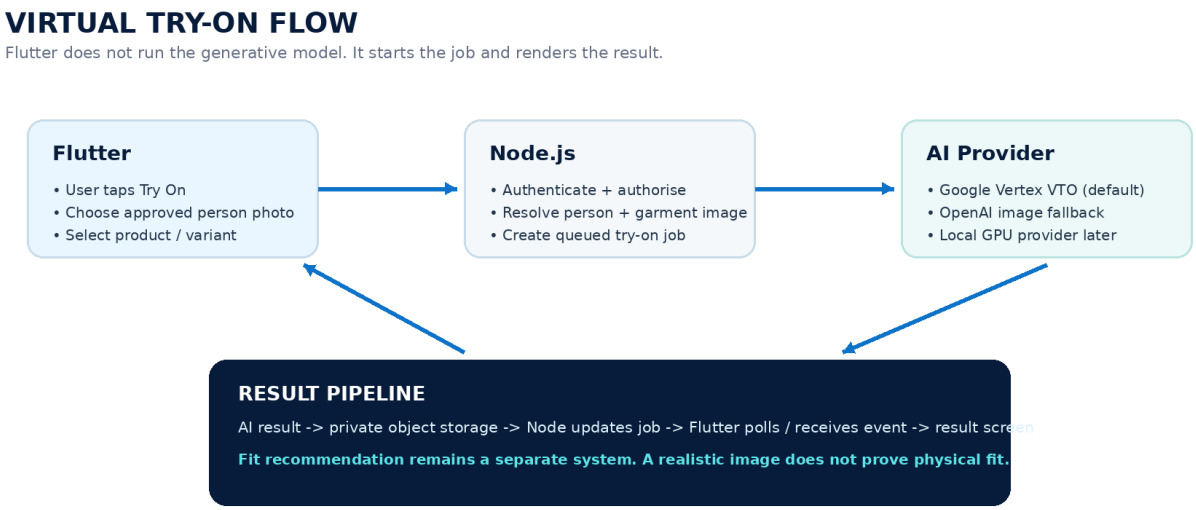
```
{
  "scanId": "scan_10021",
  "status": "completed",
  "measurements": {
    "heightCm": 175.0,
    "shoulderCm": 45.8,
    "chestCm": 101.3,
    "waistCm": 87.9,
    "hipCm": 100.6,
    "armLengthCm": 61.1,
    "inseamCm": 79.6
  },
  "confidence": { "overall": 0.89 }
}
```

Measurement acceptance

Before production, agree the supported measurements, photo protocol, reference method, pilot sample and acceptable error range. Generic image-language models may assist validation, but should not be treated as the authoritative centimetre measurement engine.

03 Virtual Try-On Flow

Try-on is an asynchronous backend job and is separate from physical fit recommendation.



Recommended public API

```
POST /api/v1/virtual-try-on
{
  "productVariantId": "variant_5001",
  "bodyProfileId": "body_101"
}

202 Accepted
{
  "jobId": "tryon_9001",
  "status": "queued"
}

GET /api/v1/virtual-try-on/tryon_9001
```

The generated image is stored privately. Node.js returns a short-lived signed URL or an authenticated media endpoint. The application never exposes the provider credential or direct provider response URL.

04 Developer Ownership Matrix

Each developer owns a defined layer; integration responsibility is explicit.

Feature	Node.js	Next.js	Flutter	Python AI
Authentication	Owner	UI/API consumer	UI/API consumer	-
Customer marketplace APIs	Owner	UI/API consumer	UI/API consumer	-
Vendor portal APIs	Owner	Owner UI	-	-
Admin APIs	Owner	Owner UI	-	-
Product/variant/stock	Owner	Vendor/admin/customer UI	Customer UI	Optional attributes
Camera/body capture	API + upload flow	Optional web capture	Owner	Validation/model
Body measurement	Job + persistence	Display	Display	Owner
Fit recommendation	Orchestration + rules	Display	Display	ML support later
Virtual try-on	Job + provider adapter	Display	Display	Local model/provider support
Payment/delivery	Owner	UI	UI	-
English/Italian	Codes + data	Owner UI	Owner UI	-
AI provider switching	Owner	No provider knowledge	No provider knowledge	Implements local endpoints
Security/audit	Owner	Frontend security	Mobile security	Internal endpoint security

Definition of ownership

Owner means responsible for design, implementation, tests and documentation in that layer. UI/API consumer means consume a stable backend contract and do not duplicate backend rules.

05 Backend Developer Standard - Node.js ESM

Domain-modular router/controller/service split with provider adapters and asynchronous jobs.

Required repository structure

```
fashion-backend/
├── src/
│   ├── app.js
│   ├── server.js
│   ├── config/
│   │   ├── env.js
│   │   ├── database.js
│   │   ├── redis.js
│   │   └── logger.js
│   ├── middleware/
│   │   ├── auth.js
│   │   ├── rbac.js
│   │   ├── validate.js
│   │   ├── rate-limit.js
│   │   └── error-handler.js
│   ├── modules/
│   │   ├── auth/
│   │   │   ├── auth.routes.js
│   │   │   ├── auth.controller.js
│   │   │   ├── auth.service.js
│   │   │   ├── auth.repository.js
│   │   │   └── auth.validation.js
│   │   ├── customers/
│   │   ├── vendors/
│   │   ├── admin/
│   │   ├── catalog/
│   │   ├── inventory/
│   │   ├── body-profiles/
│   │   ├── fit/
│   │   ├── try-on/
│   │   ├── cart/
│   │   ├── orders/
│   │   ├── payments/
│   │   ├── delivery/
│   │   ├── returns/
│   │   ├── settlements/
│   │   └── notifications/
│   ├── providers/
│   │   ├── ai/
│   │   │   ├── provider-factory.js
│   │   │   ├── measurement/
│   │   │   ├── vision/
│   │   │   ├── try-on/
│   │   │   └── assistant/
│   │   ├── payments/
│   │   ├── delivery/
│   │   └── storage/
│   ├── jobs/
│   │   ├── queues.js
│   │   ├── body-scan.worker.js
│   │   └── try-on.worker.js
│   ├── models/
│   ├── migrations/
│   ├── seeders/
│   ├── utils/
│   └── docs/openapi/
├── tests/
├── .env.example
└── package.json
```

06 Backend Rules, API Contract and Data Ownership

Business logic must remain in services; controllers are transport adapters.

Coding rules

- Node.js is the system of record; Python returns derived AI outputs and does not maintain an independent marketplace database.
- ESM imports only; no CommonJS mixing.
- Routes define paths/middleware only. Controllers parse request/response only. Services own business rules.
- Repository/data-access logic is isolated from controllers.
- Every write that spans multiple entities uses a database transaction where consistency requires it.
- All timestamps stored in UTC.
- Use stable API error codes/message keys; UI localizes user-facing text.
- Use request IDs and structured logs; never log access tokens, API keys or raw body images.
- Publish OpenAPI documentation for Flutter and Next.js integration.

Core API groups

Domain	Endpoint pattern
Identity & roles	/api/v1/auth, /customers, /vendors, /admin
Catalog	/api/v1/products, /categories, /brands
Body scan	POST /api/v1/body-scans; GET /body-scans/:id
Fit	GET /api/v1/products/:id/fit-recommendation
Try-on	POST /api/v1/virtual-try-on; GET /virtual-try-on/:jobId
Cart & checkout	/api/v1/cart, /checkout
Orders & payments	/api/v1/orders, /payments
Delivery & returns	/api/v1/shipments, /tracking, /returns, /refunds

Standard asynchronous job states

```
queued -> processing -> completed
                     -> failed
                     -> cancelled

Every job stores:
jobId, type, provider, model, userId, status,
attempts, startedAt, completedAt, failureCode,
inputAssetIds, outputAssetIds, usage/cost metadata
```

07 Next.js Developer Standard

One web codebase contains the customer website, vendor portal and administration panel.

Required repository structure

```
fashion-web/
├── src/
│   ├── app/
│   │   └── [locale]/
│   │       ├── (public)/
│   │       │   ├── page.tsx
│   │       │   ├── products/
│   │       │   └── categories/
│   │       ├── (customer)/
│   │       │   ├── account/
│   │       │   ├── body-profile/
│   │       │   ├── wishlist/
│   │       │   ├── cart/
│   │       │   └── orders/
│   │       ├── vendor/
│   │       │   ├── dashboard/
│   │       │   ├── products/
│   │       │   ├── inventory/
│   │       │   ├── orders/
│   │       │   ├── returns/
│   │       │   └── settlements/
│   │       └── admin/
│   │           ├── dashboard/
│   │           ├── vendors/
│   │           ├── customers/
│   │           ├── catalog/
│   │           ├── orders/
│   │           ├── payments/
│   │           ├── ai-operations/
│   │           └── settings/
│   ├── features/
│   │   ├── auth/
│   │   ├── catalog/
│   │   ├── body-profile/
│   │   ├── try-on/
│   │   ├── vendor/
│   │   └── admin/
│   ├── components/
│   ├── lib/
│   │   ├── api/
│   │   ├── auth/
│   │   └── validation/
│   ├── messages/
│   │   ├── en.json
│   │   └── it.json
│   └── types/
├── public/
├── tests/
└── .env.example
```

Next.js rules

- Use the App Router and keep public/customer/vendor/admin route groups clearly separated.
- All business data comes from Node.js APIs. Do not access the production database directly from Next.js.
- Never place Google/OpenAI/Python secrets in NEXT_PUBLIC_* variables.
- Prefer server-side data fetching for authenticated/SEO-sensitive pages where appropriate; use client components only when interaction requires them.
- Vendor/admin authorization is enforced again by Node.js; hiding UI is not authorization.
- Customer-facing and portal UI supports both en and it from the first release.
- Try-on and body-scan pages consume job status APIs and handle queued, processing, completed and failed states.

08 Flutter Developer Standard

Flutter owns the complete Android/iOS customer experience, including guided capture and try-on presentation.

Required repository structure

```
fashion-mobile/
├── lib/
│   ├── main.dart
│   ├── app/
│   │   ├── app.dart
│   │   ├── router.dart
│   │   └── bootstrap.dart
│   ├── core/
│   │   ├── api/
│   │   ├── auth/
│   │   ├── config/
│   │   ├── errors/
│   │   ├── storage/
│   │   ├── theme/
│   │   └── widgets/
│   ├── features/
│   │   ├── auth/
│   │   ├── home/
│   │   ├── catalog/
│   │   ├── product/
│   │   ├── body_scan/
│   │   │   ├── data/
│   │   │   ├── domain/
│   │   │   └── presentation/
│   │   ├── fit_recommendation/
│   │   ├── virtual_try_on/
│   │   ├── wishlist/
│   │   ├── cart/
│   │   ├── checkout/
│   │   ├── orders/
│   │   ├── delivery/
│   │   ├── returns/
│   │   ├── notifications/
│   │   └── profile/
│   └── l10n/
│       ├── app_en.arb
│       └── app_it.arb
├── assets/
├── test/
├── integration_test/
└── .env.example
```

Flutter rules

- Call Node.js only; never embed Gemini, Vertex, OpenAI or Python service credentials in the application.
- Use the camera for guided front/side capture. Lightweight on-device pose/visibility checks are allowed only for capture quality.
- Upload images through the backend-approved flow; respect compression, orientation and metadata requirements.
- Do not calculate authoritative chest/waist/hip centimetres in Dart. Display backend results and allow approved manual correction.
- Treat virtual try-on as an asynchronous job. The app shows progress, timeout/retry states and the final generated image.
- Keep customer state, network state and UI state separated. Use one approved state-management approach across the app; do not mix patterns per feature.
- All strings use ARB localization resources for English and Italian.
- Sensitive tokens use secure device storage; never write tokens or body-image URLs to debug logs.

09 Python AI Developer Standard

The Python service is private, model-focused and replaceable without changing the customer applications.

Required repository structure

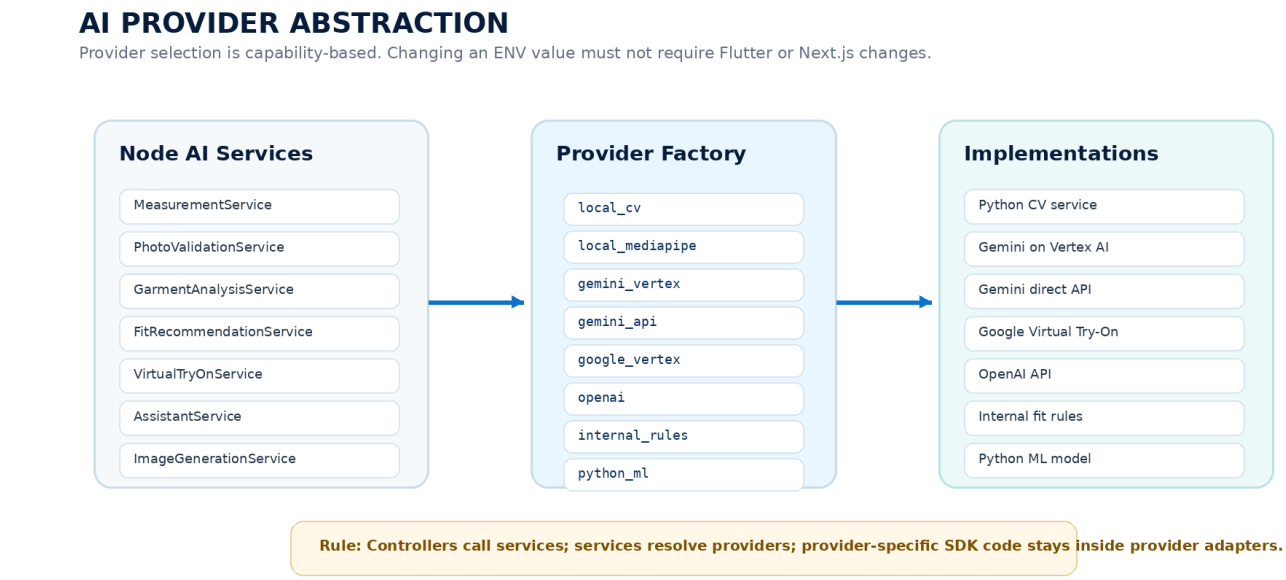
```
fashion-ai/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── health.py
│   │   ├── measurement.py
│   │   ├── fit.py
│   │   └── try_on.py
│   ├── core/
│   │   ├── config.py
│   │   ├── security.py
│   │   └── logging.py
│   ├── measurement/
│   │   ├── quality.py
│   │   ├── calibration.py
│   │   ├── pose.py
│   │   ├── segmentation.py
│   │   ├── geometry.py
│   │   └── confidence.py
│   ├── fit/
│   │   ├── rules.py
│   │   ├── features.py
│   │   └── ml_ranker.py
│   ├── try_on/
│   │   ├── preprocessing.py
│   │   ├── provider.py
│   │   └── postprocessing.py
│   ├── models/
│   ├── workers/
│   └── schemas/
├── tests/
├── scripts/
├── requirements.txt / pyproject.toml
└── .env.example
```

AI implementation rules

- FastAPI endpoints are internal/private and authenticated with a service credential or private network identity.
- Body measurement starts with deterministic calibration + pose + segmentation + geometry; model outputs include confidence.
- Do not hard-code non-commercial research checkpoints into production. Commercial licensing must be reviewed before deployment.
- Fit recommendation begins with transparent measurement rules and ease/fit allowances; ML ranking can be added when real return/fit feedback exists.
- Virtual try-on implementation is provider-based: cloud API for MVP, optional local GPU implementation later.
- Every model/provider call returns machine-readable failure codes and timing/usage metadata to Node.js.
- Python must not create marketplace users, orders, payments or vendor data.

10 AI Provider Strategy and ENV Switching

Different AI capabilities may use different providers. One global AI provider is not sufficient.



Preferred cloud pattern: use a client-owned Google Cloud project with Vertex AI for Gemini and Virtual Try-On, with OpenAI kept as an optional secondary provider. Provider and model names remain configurable.

Recommended initial provider map

Capability	Initial value	Notes
Body measurement	local_cv	Own Python CV pipeline; CPU first, GPU only if advanced models require it
Photo validation	local_mediapipe or gemini_vertex	Use local checks for cheap capture guidance; cloud validation when needed
Garment understanding	gemini_vertex	Classification/attributes; provider remains configurable
Product attribute extraction	gemini_vertex	Structured product metadata assistance
Exact fit/size	internal_rules	Own deterministic fit engine first
Virtual try-on	google_vertex	Dedicated virtual try-on service for MVP
Customer assistant	gemini_vertex	Shopping/support assistant through Node.js
Generic image editing	openai or gemini_vertex	Optional capability; keep model name configurable

11 Backend ENV Standard - AI and Provider Configuration

Commit .env.example only. Real secrets live in secret management or protected deployment configuration.

```
# ============================================================
# AI MASTER CONFIGURATION
# ============================================================
# Possible: true | false
AI_ENABLED=true

# Possible: local | cloud | hybrid
AI_MODE=hybrid

# ============================================================
# BODY MEASUREMENT
# ============================================================
# Possible: local_cv | external_api
AI_MEASUREMENT_PROVIDER=local_cv

# Internal Python service. Do not expose publicly.
PYTHON_AI_BASE_URL=http://127.0.0.1:8090
PYTHON_AI_INTERNAL_SECRET=
AI_REQUEST_TIMEOUT_MS=120000

# ============================================================
# CUSTOMER PHOTO VALIDATION
# ============================================================
# Possible: local_mediapipe | gemini_vertex | gemini_api | openai
AI_PHOTO_VALIDATION_PROVIDER=local_mediapipe

# ============================================================
# GARMENT / PRODUCT ANALYSIS
# ============================================================
# Possible: gemini_vertex | gemini_api | openai | local
AI_GARMENT_ANALYSIS_PROVIDER=gemini_vertex

# Possible: true | false
AI_PRODUCT_ATTRIBUTE_EXTRACTION_ENABLED=true

# ============================================================
# FIT / SIZE RECOMMENDATION
# ============================================================
# Possible: internal_rules | python_ml
AI_FIT_PROVIDER=internal_rules
AI_FIT_PREFERENCE_ENABLED=true
AI_FIT_CONFIDENCE_ENABLED=true
```

12 Backend ENV Standard - Cloud AI, Jobs and Storage

Cloud provider secrets are backend-only. Do not duplicate these variables in Flutter or public web configuration.

```
# ============================================================
# VIRTUAL TRY-ON
# ============================================================
# Possible: google_vertex | openai | local
AI_TRYON_PROVIDER=google_vertex
GOOGLE_VIRTUAL_TRYON_MODEL=virtual-try-on-001
AI_TRYON_IMAGE_COUNT=1
AI_TRYON_STORE_RESULTS=true

# ============================================================
# CUSTOMER ASSISTANT
# ============================================================
# Possible: gemini_vertex | gemini_api | openai | local
AI_ASSISTANT_PROVIDER=gemini_vertex
GEMINI_ASSISTANT_MODEL=
OPENAI_ASSISTANT_MODEL=

# ============================================================
# GOOGLE CLOUD / VERTEX AI
# ============================================================
GOOGLE_CLOUD_PROJECT=
# Example: europe-west1 | europe-west4 | us-central1
GOOGLE_CLOUD_LOCATION=europe-west1
# Local development only when using a credential file.
GOOGLE_APPLICATION_CREDENTIALS=
# Possible: true | false
GOOGLE_GENAI_USE_VERTEXAI=true

# Direct Gemini API only when gemini_api is selected.
GEMINI_API_KEY=
GEMINI_VISION_MODEL=
GEMINI_PRODUCT_MODEL=

# ============================================================
# OPENAI - OPTIONAL SECONDARY PROVIDER
# ============================================================
OPENAI_API_KEY=
OPENAI_PROJECT_ID=
OPENAI_VISION_MODEL=
OPENAI_ASSISTANT_MODEL=
OPENAI_IMAGE_MODEL=

# ============================================================
# FALLBACK
# ============================================================
AI_FALLBACK_ENABLED=true
# Possible: gemini_vertex | gemini_api | openai | local
AI_FALLBACK_PROVIDER=openai
AI_MAX_RETRIES=2
```

13 Backend ENV Standard - Queue, Storage and Privacy

Image handling is part of the security design, not only an infrastructure detail.

```
# ============================================================
# JOB PROCESSING
# ============================================================
AI_QUEUE_ENABLED=true
REDIS_URL=
AI_JOB_CONCURRENCY=2

# ============================================================
# IMAGE / OBJECT STORAGE
# ============================================================
# Possible: s3 | gcs | local
AI_STORAGE_PROVIDER=s3
# Possible: private | public
AI_STORAGE_VISIBILITY=private

AWS_REGION=
AWS_ACCESS_KEY_ID=
AWS_SECRET_ACCESS_KEY=
AWS_S3_BUCKET=

GCS_BUCKET_NAME=

# ============================================================
# BODY IMAGE PRIVACY
# ============================================================
AI_STORE_BODY_IMAGES=true
AI_BODY_IMAGE_RETENTION_DAYS=30
AI_STORE_TRYON_IMAGES=true
# Never true without explicit legal/customer consent.
AI_ALLOW_TRAINING_WITH_USER_IMAGES=false

# ============================================================
# LOCALIZATION
# ============================================================
APP_SUPPORTED_LANGUAGES=en,it
# Possible: en | it
APP_DEFAULT_LANGUAGE=en
```

ENV implementation requirement

Create a typed config module that validates all required variables at startup. Provider-specific variables are required only when that provider is selected. The application must fail fast with a clear configuration error instead of silently falling back to insecure defaults.

14 Client Access and Information Checklist

Request accounts and IAM access - not credentials pasted into chat or source files.

Area	Client must provide / confirm
Google Cloud / Vertex AI	Client-owned project ID; billing; Vertex AI enabled; approved region; service account/workload identity; optional GCS bucket
Gemini direct API - optional	Google AI Studio project and secure GEMINI_API_KEY only if direct Gemini API is selected instead of Vertex
OpenAI - optional fallback	Client-owned OpenAI project; billing; project/service-account API key; usage limits
Object storage	AWS S3 or GCS account/bucket, region, IAM policy and retention requirements
Payment gateway	Provider account, test/live credentials, webhook setup, settlement/refund rules
Delivery/courier	Provider account, API documentation/credentials, test environment and tracking webhook rules
Push notifications	Firebase project for Android; Apple push configuration for iOS as required
Email / SMS	Approved provider account, sender identity/domain, templates and credentials
App stores	Apple Developer and Google Play Console access
Domains / DNS	Production domains/subdomains, SSL/DNS access and email sender DNS records

Business/AI decisions required

- Exact body measurements required and acceptable pilot tolerance.
- Approved capture protocol: front/side photos, manual height, reference marker/ruler.
- Initial garment categories and mandatory garment measurements by category/size.
- Fit preferences: slim, regular, relaxed, oversized; fabric-stretch handling.
- Whether incompatible products are hidden, warned or simply ranked lower.
- Body-image retention, deletion, face visibility/masking and model-training consent.
- Virtual try-on result retention and sharing policy.
- Vendor commission, settlement, return/refund and delivery ownership rules.

15 Core Data Model and Storage Boundaries

The database schema should support multi-vendor commerce and auditable AI workflows from the start.

Domain	Suggested entities
Identity	accounts, roles, account_roles, sessions/refresh_tokens, addresses
Marketplace	vendors, vendor_users, brands, categories, products, product_variants, product_media
Sizing	garment_measurements, fit_profiles, inventory
Customer AI	body_profiles, body_scans, body_measurements, fit_recommendations, tryon_jobs
Commerce	carts, cart_items, orders, vendor_orders, order_items
Money	payments, payment_events, refunds, commissions, settlements
Fulfilment	shipments, tracking_events, returns, return_items
Operations	notifications, ai_jobs, audit_logs, system_settings

Storage rules

- Product images may be public/CDN-delivered after moderation.
- Body-scan originals and try-on source images are private by default.
- Generated try-on results are private unless the product explicitly supports sharing.
- Store object IDs/keys in SQL, not large image blobs.
- Use short-lived signed URLs or authenticated media endpoints.
- Keep AI job inputs/outputs traceable through asset IDs rather than copying URLs across tables.

16 English and Italian Localization Standard

Localization is a product requirement, not a post-launch translation task.

Frontend approach

Layer	Required approach
Next.js	Locale route or middleware strategy; messages/en.json and messages/it.json; no hard-coded customer/vendor/admin strings
Flutter	ARB resources app_en.arb and app_it.arb; all visible text via localization keys
Node.js	Return stable errorCode/messageKey where practical; store business data independently of UI language
Database content	Categories/system content support en/it fields or translation tables; vendor-authored descriptions follow the agreed content policy

Example stable backend error

```
{
  "success": false,
  "errorCode": "BODY_SCAN_POSE_INVALID",
  "messageKey": "bodyScan.poseInvalid",
  "details": { "retryable": true }
}
```

Do not use runtime AI translation for core interface labels. Static reviewed translations keep the product consistent and avoid unnecessary cost/latency.

17 Local Development and GPU Requirements

A GPU is not required on the Node.js backend server. It is only needed for selected self-hosted AI workloads.

Component	GPU?	Guidance
Node.js backend	No	Normal CPU server is sufficient.
Next.js	No	Normal web build/runtime.
Flutter	No	Android/iOS development device/emulator; no server GPU.
Python basic CV	Usually no	OpenCV, reference detection, basic pose and geometry can start on CPU.
Advanced segmentation / 3D	Recommended	GPU improves development and inference speed.
Self-hosted virtual try-on	Yes	Generative vision models are GPU-heavy.
Cloud Gemini / Vertex / OpenAI	No local GPU	Provider owns the GPU infrastructure.

If self-hosting later

- 16 GB VRAM: workable for smaller/optimized experiments, but capacity can become limiting.
- 24 GB VRAM: practical development target for many local vision/try-on experiments.
- 32 GB VRAM: preferred local workstation target when the AI developer regularly runs larger models.
- 48 GB-class datacenter GPU: suitable starting point for production self-hosted inference when concurrency and model size justify it.

MVP recommendation

Do not buy a permanent GPU only to launch the MVP. Use local CPU for the measurement PoC where possible and cloud AI for virtual try-on. Add a GPU worker later only when model control, cost or volume justifies it.

18 Security, Privacy and AI Cost Controls

Body images and AI generation require stronger controls than ordinary catalogue images.

Security baseline

- TLS for all public traffic; private network or authenticated service-to-service calls for Python AI.
- RBAC enforced by Node.js for customer, vendor and admin roles.
- Access/refresh token strategy appropriate for mobile; secure HttpOnly cookie strategy for web where used.
- Secrets are never committed; use deployment secret management and separate dev/UAT/prod credentials.
- Signed object URLs have short expiry; private storage buckets reject public listing.
- Validate file type, size and image dimensions; strip unnecessary metadata where practical.
- Rate-limit body scans and try-on generation by user/account/IP and enforce business quotas.
- Verify payment and delivery webhooks using provider signatures.
- Audit admin, vendor approval, refund, settlement and AI provider changes.

AI privacy defaults

- AI_ALLOW_TRAINING_WITH_USER_IMAGES=false by default.
- Define automatic deletion/retention policy for original body images.
- Allow customer deletion of body profile and associated media according to policy.
- Do not log raw image URLs if logs are broadly accessible.
- Capture explicit consent before body-scan/try-on processing.

19 Parallel Development Plan

The team should not wait for the AI model before completing the application flows.

Delivery target

Plan the agreed MVP across approximately 90-100 working days (about 4-5 months), with web, mobile, backend and AI workstreams progressing in parallel. Final dates depend on client approvals, third-party access and AI pilot acceptance.

#	Workstream	Outcome
1	Backend contract	Define database, OpenAPI schemas, auth, uploads and mock AI endpoints.
2	Mock AI	Return deterministic sample body measurements, fit results and try-on job states.
3	Next.js	Build customer, vendor and admin interfaces against stable Node APIs.
4	Flutter	Build complete customer app, guided scan flow and try-on result flow against mocks.
5	Python AI	Build body measurement pipeline and private FastAPI contracts.
6	Cloud providers	Integrate Gemini/Vertex/OpenAI adapters behind Node provider factory.
7	Integration	Replace mocks with real workers without changing frontend contracts.
8	Commerce	Complete payment, delivery, return/refund and settlement flows.
9	QA/UAT	Cross-platform, AI pilot, security and failure-path testing.
10	Launch	Production migration, stores, monitoring, backups and handover.

API-first integration rule

Backend publishes request/response schemas before frontend implementation. Breaking changes require versioned contract review. Flutter and Next.js must not depend on undocumented response fields.

20 Testing, Quality Gates and Definition of Done

A feature is not complete when the happy-path screen works; it is complete when the full contract, error states and tests pass.

Required testing by team

Team	Minimum quality gate
Node.js	Unit service tests; integration/API tests; auth/RBAC; transaction rollback; webhooks; queues; provider fallback; migrations
Next.js	Component/page tests where valuable; API error states; RBAC navigation; en/it coverage; responsive layouts
Flutter	Unit/widget/integration tests; camera permissions; upload interruption; offline/error handling; en/it; Android/iOS device checks
Python AI	Deterministic preprocessing tests; measurement regression set; confidence/failure cases; model/provider benchmarks; licensing record
End-to-end	Register -> scan -> fit -> try-on -> cart -> payment -> order -> delivery/return; vendor/admin flows

Definition of Done

- Code reviewed and merged with no unresolved critical comments.
- API/OpenAPI contract updated.
- Database migration included where schema changed.
- English and Italian strings complete.
- Loading, empty, permission denied, retryable failure and non-retryable failure states implemented.
- No secrets or debug credentials committed.
- Logs/metrics added for operationally important paths.
- Automated tests and manual UAT scenario pass.
- Deployment notes and ENV changes documented.

21 Repository, Git and Handover Standards

The four repositories should remain independently deployable but coordinated through versioned contracts.

Repositories

```
fashion-backend   -> Node.js ESM public API and orchestration
fashion-web       -> Next.js customer + vendor + admin
fashion-mobile    -> Flutter Android + iOS customer app
fashion-ai        -> Python internal AI service / local model workers
```

Git standards

- Protect main/production branches and require reviewed pull requests.
- Use small feature branches with clear scope; avoid mixing unrelated refactors with business features.
- Every pull request states API/schema/ENV impact and test evidence.
- Never commit .env, provider keys, service-account JSON, signing keys or production certificates.
- Tag coordinated releases and record compatible backend/mobile/web versions when contracts change.
- Use dev, UAT/staging and production environments with separate credentials and databases.

Handover artifacts

- README with local setup and deployment commands for each repository.
- Complete .env.example with comments and provider-specific required fields.
- OpenAPI specification and sample requests for Flutter/Next.js.
- Database migration history and seed instructions.
- AI model/provider inventory with licence and version notes.
- App-store build/release instructions.
- Cloud/IAM, backup, monitoring and incident notes.

22 Developer Start Checklist

Use this page as the kickoff checklist before coding begins.

Owner	Before implementation
Backend	Repository created; SQL/Redis ready; API v1 convention agreed; OpenAPI baseline; provider factory skeleton; object storage/upload flow; migrations enabled.
Next.js	Route groups created; auth/RBAC integration plan; en/it resources; API client; customer/vendor/admin layouts; no provider secrets.
Flutter	Project flavours/environments; secure storage; API client; routing/state pattern agreed; en/it ARB; camera permissions/capture PoC; no provider secrets.
Python AI	FastAPI internal auth; measurement schema; sample images/test set; calibration PoC; model licence review; CPU baseline; job response contract.
Project Lead	Client access checklist sent; AI/privacy decisions tracked; third-party provider ownership confirmed; UAT acceptance criteria documented.

Team rule

When a requirement is unclear, update the shared API/business decision document before independently implementing different assumptions in each repository.

A Provider Notes and Current Configuration References

Keep model identifiers in ENV because cloud provider offerings and versions change over time.

Current implementation guidance at the date of this document uses Google Vertex AI as the preferred cloud ecosystem for Gemini-based vision/assistant tasks and the dedicated Virtual Try-On service. OpenAI remains an optional secondary provider for multimodal reasoning and image workflows.

Current reference points

Provider area	Implementation note	Reference
Google Virtual Try-On	The generally available Vertex AI endpoint is virtual-try-on-001.	https://docs.cloud.google.com/vertex-ai/generative-ai/docs/release-notes
Gemini API authentication	Direct Gemini API clients may use GEMINI_API_KEY / GOOGLE_API_KEY; production may instead use Vertex AI identity.	https://ai.google.dev/gemini-api/docs/api-key
OpenAI image models	OpenAI image generation/editing model names should be selected through OPENAI_IMAGE_MODEL rather than hard-coded.	https://developers.openai.com/api/docs/models
OpenAI project credentials	Prefer a client-owned project/service account key with appropriate scopes/expiry rather than a personal developer key.	https://developers.openai.com/api/reference

These links are implementation references, not contractual provider commitments. The selected region, model availability, pricing, quotas and legal terms must be checked again at production onboarding.

23 Technical Sign-Off

The following sign-off confirms that the development team understands the architecture boundaries and implementation standards in this guide.

Backend Lead	**Next.js Lead**
Name: _____	Name: _____
Date: _____	Date: _____
Flutter Lead	**Python / AI Lead**
Name / Date: _____	Name / Date: _____

Cyber Impulse Software Solutions

Technical Architecture & Developer Implementation Guide - Version 1.0